

OpenAI Responses API를 활용한 서비스 구현 튜토리얼

본 튜토리얼에서는 OpenAI **Responses API**를 활용하여 AI 기반 서비스를 개발하는 방법을 단계별로 설명합니다.

1. OpenAI Responses API 소개

Responses API는 OpenAI가 제공하는 최신 API로, 모델에게 역할과 질문을 함께 전달하면 바로 답변을 받을 수 있습니다. 다음과 같은 특징이 있습니다.

- **대화 지속성**: 직전 응답의 `response.id`를 `previous_response_id`로 넘기기만 하면, 별도의 저장 공간을 만들지 않고도 이전 대화 맥락이 자동으로 이어집니다.
- **다양한 도구 통합**: 코드 실행(Code Interpreter), 문서 검색(File Search), 웹 검색(Web Search), 함수 호출(Function Calling) 등의 도구를 모델에 연결할 수 있습니다. 이를 통해 모델의 한계를 넘어서는 작업(예: 데이터 계산, 파일 처리, 외부 검색 등)을 자동화할 수 있습니다.
- **개인화된 지시어**: `instructions` 파라미터로 모델의 성격과 역할을 정의할 수 있습니다. 예를 들어 "당신은 친절한 고객지원 봇입니다"와 같은 지시어로 모델의 톤과 도메인 지식을 설정할 수 있습니다.
- **간결한 응답 접근**: `response.output_text`로 최종 텍스트 답변을 바로 꺼낼 수 있습니다.

요약하면, Responses API는 복잡한 대화 상태 관리, 외부 도구 통합, 문서 검색 등 많은 부분을 OpenAI 플랫폼이 맡아주므로, 개발자는 핵심 로직 구현에 집중할 수 있습니다. 이러한 이유로, 챗봇이나 자동화 에이전트를 만든다면 Responses API가 강력한 선택지가 됩니다.

>Note: API는 계속 발전하므로, 사용 전 [공식 문서](<https://platform.openai.com/docs/api-reference/responses>)를 확인하는 습관을 들이는 게 좋습니다.

2. API 키 설정 및 환경 변수 사용

OpenAI API를 사용하려면 API 키가 필요합니다. OpenAI 플랫폼의 API 키 관리 페이지에서 비밀번호를 생성할 수 있습니다. 생성된 키는 한 번만 표시되므로, 반드시 복사하여 안전한 곳에 저장하세요. 일반적으로 이 키를 소스 코드에 하드코딩하지 않고, 별도의 설정으로 관리하는 것이 좋습니다. **환경 변수(Environment Variable)**를 사용하면 API 키를 소스 코드에 노출하지 않고 관리할 수 있습니다. 개발 PC 또는 서버의 환경 변수 OPENAI_API_KEY에 키를 저장해 두면, OpenAI 라이브러리가 자동으로 이를 읽어 사용할 수 있습니다. Python 개발 환경에서는 python-dotenv 패키지를 활용해 .env 파일에 키를 저장하고 로드하는 방식이 편리합니다.

다음은 API 키를 설정하고 로드하는 과정입니다:

1. python-dotenv 설치: 터미널에서 `pip install python-dotenv` 명령으로 설치합니다 (한번만 수행).

2. 환경 변수 파일 생성: 프로젝트 루트 디렉토리에 .env 파일을 만들고, 아래와 같이 OpenAI API 키를 입력합니다 (따옴표 없이 실제 키로 대체).

```
...  
  
OPENAI_API_KEY=sk-*****  
  
...
```

3. 코드에서 로드: Python 코드에서 python-dotenv를 이용해 .env를 로드합니다. OpenAI 공식 Python SDK는 환경 변수 OPENAI_API_KEY를 자동으로 인식하므로, `OpenAI()`를 인자 없이 호출해도 내부적으로 이 값을 사용합니다.

3. 기본적인 응답 생성 및 메시지 보내기

`client.responses.create()` 한 번의 호출로 역할(instructions)과 질문(input)을 동시에 넘겨 바로 답변을 받습니다.

예를 들어 간단한 도움말 챗봇 역할을 지정해서 사용자 질문에 답변해보겠습니다.

위 코드에서는 ``client.responses.create`` 메서드를 사용해 응답을 생성했습니다. 주요 파라미터를 살펴보면:

- ****model****: 응답 생성에 사용할 언어 모델을 지정합니다.

GPT-4 계열:

- gpt-4o - 최신 멀티모달 모델
- gpt-4o-mini - 경량화된 GPT-4o (비용 효율적, 이 튜토리얼의 기본값)
- gpt-4-turbo - 더 빠르고 효율적인 GPT-4
- gpt-4 - 기본 GPT-4 모델

GPT-3.5 계열:

- gpt-3.5-turbo - 빠르고 효율적인 모델

- ****instructions****: 시스템 레벨의 지시어로, 이번 호출에서 모델이 따르게 될 기본 규칙이나 역할을 정의합니다. (기존 ChatGPT의 시스템 메시지와 같은 역할입니다.) 이 지시어는 매 호출마다 함께 넘겨줘야 합니다. 자주 쓰는 역할이라면 문자열을 변수나 함수로 만들어두고 재사용하면 편리합니다.

- ****input****: 사용자의 질문(또는 대화 내용)을 전달합니다.

- ****tools****: 활성화할 도구 목록입니다. 기본적인 Q&A에서는 특별한 툴이 필요 없으므로 생략했습니다. 나중에 코드 인터프리터나 파일 검색 등을 사용할 때 이 필드를 설정합니다.

응답이 생성되면 고유 ID(`response.id`)가 반환됩니다. 이 ID는 이후 대화를 이어갈 때 `previous\_response\_id`로 지정하면 되므로 저장해둡니다.

>🔍 참고: 각 응답(response)은 `previous\_response\_id`로 바로 이전 응답을 가리키는 **체인(사슬) 구조**를 이룹니다. OpenAI 서버가 이 체인을 따라가며 이전 대화 내용을 자동으로 반영해주기 때문에, 개발자가 대화 내역을 매번 직접 다시 보낼 필요가 없습니다.

대화 이어가기 (멀티턴)

이제 방금 받은 답변에 이어서 추가 질문을 해보겠습니다. `previous\_response\_id`에 직전 `response.id`를 넘기면, 모델이 이전 대화 맥락을 기억한 채로 답변합니다.

예제: 대화형 챗봇 (interactive chatbot)

사용자가 계속 입력하고, 종료 명령을 입력할 때까지 대화를 이어가는 챗봇을 구현해보겠습니다.

역할(instructions)을 고정해두고, 매 턴마다 `previous\_response\_id`로 직전 응답을 이어붙이며 `responses.create()`를 반복 호출하는 구조입니다. 대화가 끝나면 이력을 파일로 저장합니다.

4. 파일 업로드 및 코드 인터프리터 활용

파일 업로드는 사용자가 제공한 데이터를 모델이 활용할 수 있게 하는 중요한 기능입니다. 예를 들어 CSV 데이터를 분석하거나, 텍스트 파일 안의 숫자를 더하는 등의 작업이 가능합니다. 방식은 다음과 같습니다.

1. ``client.files.create()``로 파일을 업로드해 ``file_id``를 얻습니다.
2. ``tools=[{"type": "code_interpreter", "container": {"type": "auto", "file_ids": [file_id]}}`` 형태로 코드 인터프리터 도구에 파일을 직접 연결합니다.
3. 코드 인터프리터가 실행되면 결과와 함께 ``container_id``가 반환되며, 이 컨테이너(가상 실행 환경) 안에서 Python 코드가 실제로 실행됩니다.

>Tips: 컨테이너는 일정 시간(기본 약 1시간, 유휴 20분)이 지나면 만료됩니다. 만료된 컨테이너는 복구할 수 없으므로, 새 요청에서 파일이 다시 필요하면 ``container.file_ids``에 같은 ``file_id``를 다시 넣어주면 됩니다. (파일 자체를 재업로드할 필요는 없습니다.)

5. 파일 검색 (File Search) 활용

코드 인터프리터가 "파일을 열어 계산/가공"하는 도구라면, File Search는 "문서 여러 개를 대상으로 관련 내용을 찾아 답변"하는 도구입니다. 이를 위해 ****Vector Store****(문서를 검색 가능한 형태로 저장해두는 공간)를 먼저 만들고, 거기에 파일을 넣은 뒤, 그 Vector Store의 id를 도구에 지정합니다.

절차는 다음과 같습니다.

1. ``client.vector_stores.create()``로 Vector Store를 만듭니다.
 2. ``client.vector_stores.files.upload_and_poll()``로 파일을 업로드하고, 임베딩(embedding) 및 인덱싱이 끝날 때까지 기다립니다.
 3. ``tools=[{"type": "file_search", "vector_store_ids": [vector_store.id]}``로 지정해 질문합니다.
-